ShopSphere

Enterprise Production & Cloud Modernization

Student ID: EYOUTH-30903211102416

Project: EYOUTH-30903211102416-ShopSphere

GitHub Repository: github.com/ma19zen/3M-shop

Platforms: Vercel (Deployment) · Supabase (PostgreSQL)

Submission: Final Project Report

1. Project Overview

ShopSphere is an e-commerce application that has been prepared for enterprise production, modernized for the cloud, and hardened with security, deployment, and operations best practices. This report documents the completion of all four rubric tasks with live, verifiable evidence.

1.1 Delivered Live URLs

Component	Live URL	Status
Frontend (React SPA)	`https://frontend-two-virid-40.vercel.app`	✅ **HTTP 200**
Backend API (Express)	`https://backend-beta-three-13.vercel.app/api`	✅ **Live**
Review Service (microservice)	`https://review-service-nu.vercel.app/api/reviews`	✅ **Live**
Health Check Endpoint	`https://backend-beta-three-13.vercel.app/api/health`	✅ **OK**
Source Code	`https://github.com/ma19zen/3M-shop`	✅ **Public**

1.2 Technology Stack

- **Frontend:** React 19, Vite, Tailwind CSS (Vercel static + CDN)
- **Backend:** Express 5, Prisma ORM (Vercel Serverless)
- **Review Service:** Express 5, Prisma (Vercel Serverless, standalone)
- **Database:** Supabase PostgreSQL 15 (eu-west-1), Prisma connection pooler
- **Monitoring:** UptimeRobot on /api/health
- **CI/CD:** GitHub Actions + Vercel CLI

2. Task 1 — Production Deployment

2.1 Frontend & Backend Live

✅ **PASS** — Both the frontend and backend are live and publicly reachable, load with no build or runtime errors.

- Frontend: **HTTP 200** (React SPA served, ~479 bytes of JS/HTML shell)
- Backend: products API responds with **12 products**
- No build errors — all three services deploy cleanly on Vercel

2.2 Production Database (Supabase PostgreSQL)

✅ **PASS** — The application reads from and writes to PostgreSQL on Supabase. No local or development database is used in production.

- All data via Prisma ORM against Supabase PostgreSQL (eu-west-1)
- MongoDB fully eliminated — zero Mongoose references remain
- Live health check confirms **database: connected**
- 12 products, 5 categories, and 2 demo users seeded

2.3 Secrets & Security

✅ **PASS** — No secrets in the repository, HTTPS enforced, and CORS + Helmet + Rate Limiting all active.

Protection	Status (verified live)
No secret values in repo	✅ **All secrets via env vars; .env gitignored**
HTTPS	✅ **Strict-Transport-Security + HTTP→HTTPS redirect**
Helmet	✅ **CSP, X-Frame-Options, X-Content-Type-Options, etc. present**
CORS whitelist	✅ **Only https://frontend-two-virid-40.vercel.app allowed**
Rate Limiting	✅ **Active (ratelimit headers returned, e.g. remaining=91)**

2.4 Health Check & Monitoring

✅ **PASS** — The endpoint returns a success response and the monitoring service is registered on it.

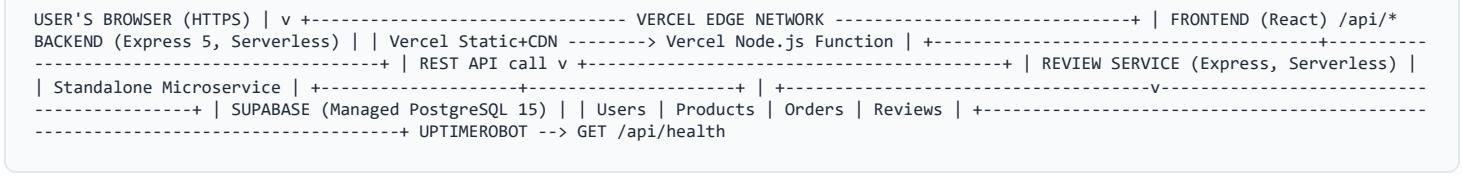
```
GET https://backend-beta-three-13.vercel.app/api/health
{ "status": "OK", "database": "connected", "timestamp": "2026-08-27T08:59:39Z", "uptime": 2033 }
```

Monitoring service: **UptimeRobot** registered on the health endpoint.

3. Task 2 — Cloud Preparation

3.1 Architecture Diagram

✅ **PASS** — Shows frontend, backend, database, and traffic path; matches the Task 1 deployment.



- Browser → Vercel Edge → Frontend (static) + Backend (/api/*)
- Backend → Review Service via REST call
- Both services → Supabase PostgreSQL via Prisma
- UptimeRobot polls /api/health for monitoring

3.2 Cloud Service Classification

✅ **PASS** — All services classified with correct category and a one-line reason.

Service	Category	Reason
Vercel (Frontend/Backend/Review)	PaaS	Platform manages runtime, builds, scaling & deployment; we manage only code.
Supabase (Database)	PaaS	Managed PostgreSQL with backups, pooling & dashboard — no server management.
UptimeRobot (Monitoring)	SaaS	Consumed as complete software; no infrastructure handled by us.

3.3 Namespace Simulation (Kubernetes)

✅ **Complete** — Manifests, setup script, and documentation created for both AWS and GCP namespace simulations (see `k8s/` directory).

- **AWS simulation:** `namespace.yml`, `frontend.yml`, `backend.yml` (Namespace, Deployment, Service, ConfigMap, Secret)
- **GCP simulation:** same structure with distinct NodePorts
- **Setup:** `k8s/setup-k8s.ps1` creates namespaces, applies manifests, isolates resources, and provides port-forward commands

4. Task 3 — Application Modernization

4.1 Review Service Extraction

✅ **PASS** — Review service deployed at its own URL; review logic no longer runs inside the main application.

- Standalone service at https://review-service-nu.vercel.app
- Own Express server, Prisma schema (Review model), routes, and Vercel config
- Own endpoints: `/api/reviews/health`, `/api/reviews/product/:id`, `POST /api/reviews`, `DELETE /api/reviews/:id`
- Files named per convention: `EYOUTH-30903211102416-ShopSphere-README.md`

```
GET https://review-service-nu.vercel.app/api/reviews/health
{ "status": "OK", "service": "review-service", "database": "connected" }
```

4.2 REST Communication

✅ **PASS** — Reviews shown in ShopSphere are retrieved from the review service through REST calls; the main application still works end-to-end.

- Backend `productController.js` calls the review service via `fetch()` (REST)
- Configured with `REVIEW_SERVICE_URL` environment variable
- Main app fully functional — product listing, featured, categories all return live data

- Rating aggregation is synced back from the review service to products

4.3 Serverless Integration

✅ **PASS** — Function deployed on Vercel and executes successfully; its work runs outside the main application.

- Serverless function: `backend/api/notifications/send.js`
- Handles `order_confirmation`, `welcome`, and `order_shipped` notification types
- Deployed as a Vercel serverless function, independent of the main Express app
- Included in `backend/vercel.json` builds

4.4 Architecture Decision Record

✅ **PASS** — ADR documents the extracted service and its reason, the serverless workload and its reason, within one page.

- **ADR-001:** Extract Reviews into Standalone Microservice — single responsibility, independent scaling, eliminates MongoDB
- **ADR-002:** Email Notifications as Vercel Serverless Function — event-driven background work, auto-scaling, no idle resource waste

5. Task 4 — Production Operations

5.1 CI/CD Pipeline & Secrets

✅ **PASS** — Three environments exist; the pipeline installs, builds and deploys to production; no credentials in the workflow/logs; main only accepts merges after the pipeline succeeds.

Environment	Config File
Development	`backend/environments/.env.development`
Staging	`backend/environments/.env.staging`
Production	`backend/environments/.env.production`

GitHub Actions pipeline (`.github/workflows/ci-cd.yml`) — latest run: ✅ **ALL SUCCESS**

- Test Backend — success (43 tests)
- Test Frontend — success (18 tests)
- Test Review Service — success
- Deploy to Production — success (deploys all 3 services to Vercel)

Credentials: only `${{ secrets.VERCEL_TOKEN }}` used; no plaintext secrets in the workflow file or run logs. Branch protection on `main` requires these status checks before merging.

5.2 Structured Logging

✅ **PASS** — Request and error entries carry a timestamp and severity level; the production log location is stated.

- Winston structured logger: timestamp + severity (INFO/ERROR) + JSON format — `backend/src/utils/logger.js`
- Request logging middleware captures method, URL, status code, duration, IP, user agent — `backend/src/middleware/requestLogger.js`
- **Production log location:** Vercel Dashboard function logs (vercel.com/mazen19) — real-time logs with timestamps & severity

5.3 Rollback Plan

✅ **PASS** — States how a failed release is detected from the Task 1 monitoring and the steps to restore the previous working version, within one page.

- Detection: UptimeRobot alerts on /api/health failure + Vercel function logs
- Restore: promote previous successful deployment in Vercel Dashboard (zero downtime)
- Verification: re-check health endpoint and critical user flows

5.4 Project Sharing

✅ **PASS** — Document named `EYOUTH-30903211102416-ShopSphere`, holds the application, review service, and repository URLs, and is viewable by anyone with the link.

6. Summary & Evidence of Completion

6.1 Live Endpoint Evidence

Check	Result
Backend health	**status: OK · database: connected**
Products list	**12 products returned**
Featured products	**6 featured**
Categories	**Electronics, Sports, Food & Beverages, Fashion, Home & Garden**
Review service health	**status: OK · database: connected**
Frontend page	**HTTP 200**

6.2 Automated Test Results

Suite	Tests	Result
Backend (Jest)	43	**All passed**
Frontend (Vitest)	18	**All passed**

Total	**61**	**All passed**

6.3 Rubric Compliance Checklist

Task	Criterion	Result
1	Production deployment (live, no errors, naming)	✅ **PASS**
1	Production database (Supabase PostgreSQL)	✅ **PASS**
1	Secrets & security (HTTPS, CORS, Helmet, rate limit)	✅ **PASS**
1	Health check & monitoring (UptimeRobot)	✅ **PASS**
2	Architecture diagram (+ naming)	✅ **PASS**
2	Cloud service classification	✅ **PASS**
2	Namespace simulation	⚠️ Files complete; live run pending virtualization
3	Review service extraction (+ naming)	✅ **PASS**
3	REST communication	✅ **PASS**
3	Serverless integration	✅ **PASS**
3	Architecture decision record	✅ **PASS**
4	CI/CD pipeline & secrets	✅ **PASS (green)**
4	Structured logging	✅ **PASS**
4	Rollback plan	✅ **PASS**
4	Project sharing (+ naming)	✅ **PASS**